```
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $python3 basic_hashing.py
==================================================
BASIC HASHING DEMONSTRATION
==================================================
Original password: MySecretPassword123

MD5 Hash (insecure):
Length: 32 characters
Hash: d87493fe65f0e03816fff836b75b858e

SHA-256 Hash (secure):
Length: 64 characters
Hash: b169822e0e15ac5a2cccc3f740d5f263b501003d183928fc434653ecebd744bc

Same password produces SAME hash:
SHA-256 (first):  b169822e0e15ac5a2cccc3f740d5f263b501003d183928fc434653ecebd744bc
SHA-256 (second): b169822e0e15ac5a2cccc3f740d5f263b501003d183928fc434653ecebd744bc
```

```
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $python3 password_cracking_demo.py
==================================================
PASSWORD CRACKING DEMONSTRATION
==================================================
Target password: password123
MD5 hash in database: 482c811da5d5b4bc6d497ffa98491e38
SHA-256 hash in database: ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f38
3d4473e94f

Simulating dictionary attack with common passwords...

Attempting to crack MD5 hash:
✓ CRACKED! Password found: password123
Time taken: 0.0000 seconds

Attempting to crack SHA-256 hash:
✓ CRACKED! Password found: password123
Time taken: 0.0000 seconds
```

```
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $python3 salting_demo.py
==================================================
SALTING DEMONSTRATION
==================================================
Password: MySecretPassword123
Salt1 (hex): 1bb03dcb01c10132b3cf669c5976fdcb
Salt2 (hex): ec5e6b68bef6277ea47021fb80f80904

Salted Hash1: 30d3637a31d3862f2925ae8cf65e4c794447bb00f4edc9518a838f109f139c16
Salted Hash2: f5d960ed8e806df6dc375ffe8a7edd5b751ef0f04d2200d009ec50aa85a63ebb

Same password + different salts = DIFFERENT hashes!
This prevents rainbow table attacks.

==================================================
PASSWORD VERIFICATION SIMULATION
==================================================
User registered with password: user123
Stored salt: 71889a5777ce6511...
Stored hash: c923ebc5f9b13ae0f8525c4f9122eafc...

✗ Login FAILED with password: wrongpass
✓ Login SUCCESS with password: user123
```


```
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $python3 salting_demo.py
==================================================
SALTING DEMONSTRATION
==================================================
Password: MySecretPassword123
Salt1 (hex): 1bb03dcb01c10132b3cf669c5976fdcb
Salt2 (hex): ec5e6b68bef6277ea47021fb80f80904

Salted Hash1: 30d3637a31d3862f2925ae8cf65e4c794447bb00f4edc9518a838f109f139c16
Salted Hash2: f5d960ed8e806df6dc375ffe8a7edd5b751ef0f04d2200d009ec50aa85a63ebb

Same password + different salts = DIFFERENT hashes!
This prevents rainbow table attacks.

==================================================
PASSWORD VERIFICATION SIMULATION
==================================================
User registered with password: user123
Stored salt: 71889a5777ce6511...
Stored hash: c923ebc5f9b13ae0f8525c4f9122eafc...

✗ Login FAILED with password: wrongpass
✓ Login SUCCESS with password: user123
```


```
==================================================
SECURE PASSWORD STORAGE SYSTEM
==================================================
1. Register User
2. Login
3. List Users
4. Exit

Enter your choice (1-4): 2
Enter username: Aneesh Sangran
Enter password: aneesh@123

Login successful!

==================================================
SECURE PASSWORD STORAGE SYSTEM
==================================================
1. Register User
2. Login
3. List Users
4. Exit
```

```
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $python3 performance_comparison.py
============================================================
HASHING ALGORITHM PERFORMANCE COMPARISON
============================================================

Benchmarking with 100,000 iterations each...
------------------------------------------------------------
MD5      : 0.0643 seconds (1,555,278 hashes/sec)
SHA-1    : 0.0655 seconds (1,527,256 hashes/sec)
SHA-256  : 0.0669 seconds (1,495,466 hashes/sec)
SHA-512  : 0.0990 seconds (1,010,364 hashes/sec)
VMware: No 3D enabled (0, Success).
libEGL warning: egl: failed to create dri2 screen
VMware: No 3D enabled (0, Success).
libEGL warning: egl: failed to create dri2 screen

✓ Performance graph saved as 'hash_performance.png'
```

```
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $python3 rainbow_table_demo.py
============================================================
RAINBOW TABLE ATTACK DEMONSTRATION
============================================================

1. Attacker steals database with UNSALTED hashes:
   user1: 482c811da5d5b4bc...
   user2: d8578edf8458ce06...
   user3: 21232f297a57a5a7...
   user4: 0d107d09f5bbe40c...

2. Attacker uses pre-computed rainbow table:
   Rainbow table contains 8 pre-computed hashes

3. Attacker cracks passwords instantly:
   ✓ user1: password = 'password123'
   ✓ user2: password = 'qwerty'
   ✓ user3: password = 'admin'
   ✓ user4: password = 'letmein'

   Cracked 4 out of 4 passwords instantly!
```

```
1. Database stores salted hashes:
   user1: salt=8ddfc7bb, hash=59cf48fee35deb07...
   user2: salt=948fb9dc, hash=0d23a58fe0f95dd8...
   user3: salt=453862b6, hash=a4f367ae1d514e0d...
   user4: salt=050c5c2c, hash=7d582f39e75c6e35...

2. Attacker's rainbow table is useless because:
   - Each password has a unique salt
   - Would need separate rainbow table for each salt
   - Computationally impossible for all possible salts
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $echo -n "password123" | openssl md5
MD5(stdin)= 482c811da5d5b4bc6d497ffa98491e38
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $echo -n "password123" | openssl sha256
SHA2-256(stdin)= ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $echo -n "salt123password123" | openssl sha256
SHA2-256(stdin)= c465f66c6ac481a7a17e9ed5b4e2e7e7288d892f12bf1c95c140901e9a70436e
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $openssl rand -hex 16
201d654fe8d38254ad9597da74000fb3
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $echo "My secret data" > testfile.txt
┌─[attacker@parrot]─[~/Downloads/lab]
└─► $openssl dgst -sha256 testfile.txt
SHA2-256(testfile.txt)= 9f55972f3b2bd31cb80be36c0175cf68cb8408d9bede9b40a2cafaa59e37feca
┌─[attacker@parrot]─[~/Downloads/lab]
```